

Optimisation of the TOV and Tidal-Deformability Solver

Performance improvements to `logtov_seq_geom_tidal.f90`

A technical record of the changes made to accelerate the neutron-star structure and tidal-deformability calculation while preserving the numerical output of the original solver.

Author: Ayush Fotedar
Code: `logtov_seq_geom_tidal.f90`
Language: Fortran
Compiler: GNU Fortran (`gfortran`)
Date: October 1, 2026

Contents

1 Purpose	2
2 Original Numerical Structure	2
3 Benchmark and Validation Strategy	3
4 Compiler Optimisation Tests	3
4.1 -O1	3
4.2 Why -O2 and -O3 were rejected	4
5 Source-Level Optimisations	4
5.1 Use the actual EoS table length	4
5.2 Replace linear EoS searches with binary searches	4
5.3 Cache $\epsilon(P)$ in <code>rhsp</code>	5
5.4 Cache the base $\epsilon(P)$ evaluation in <code>rhsb</code>	5
5.5 Disable unnecessary diagnostic output	6
5.6 Cache repeated identical EoS queries in <code>EosShen2D</code>	6
6 Performance Summary	7
7 What Was Deliberately Not Changed	8
8 Numerical Equivalence Checks	8
9 Why the Original Code Was Slow	9
10 Full-Sequence Behaviour	9
11 Constraints and Remaining Numerical Issues	10
11.1 Monotonic EoS requirement	10
11.2 Derivative-sensitive observables	10
11.3 Known correctness issue kept separate from optimisation	10
12 Final Recommended Build	10
13 Optimisation Timeline	11
14 Conclusion	11

1 Purpose

The original Fortran program solves the Tolman–Oppenheimer–Volkoff (TOV) stellar-structure equations together with the tidal perturbation equations for a sequence of central energy densities. For each stellar model, six first-order differential equations are evolved simultaneously with a fourth-order Runge–Kutta (RK4) scheme.

The optimisation work had one primary objective:

Reduce execution time without changing the physical equations, the radial integration step, the EoS interpolation rule, or the primary TOV/tidal output.

The optimisation was therefore performed incrementally. After each source-code change, the same benchmark sequence was rerun and the resulting `logtov_seq_geom_tidal.dat` file was compared with the previous accepted version.

The final accepted source-level changes reduced the representative 20-model runtime from approximately

$$11.46 \text{ s}$$

to

$$1.63 \text{ s},$$

corresponding to a speed-up of approximately

$$7.0\times$$

or a wall-clock reduction of about

$$85.8\%.$$

2 Original Numerical Structure

For every central density, the program performs the following sequence:

$$\text{EoS interpolation} \rightarrow \text{RK4 stellar integration} \rightarrow M, R \rightarrow k_2, \Lambda$$

The RK4 integration repeatedly evaluates routines including `rhsp`, `rhsm`, `rhsn`, `rhsr`, `rhsh`, and `rhsb`. These routines in turn request thermodynamic quantities from the equation of state many times during every radial step.

The most important interpolation directions are

$$\epsilon = \epsilon(P)$$

through `eps(pp)` / `EosShen2D`, and

$$P = P(\epsilon)$$

through `pout(ee)` / `poutshen`.

The EoS interpolation itself remains a linear interpolation in $\log_{10}$ -space. The optimisation changes described below alter how the surrounding EoS points are *found* and how repeated evaluations are reused; they do not replace the interpolation formula.

3 Benchmark and Validation Strategy

A representative 20-model sequence was used while optimising the solver. The benchmark EoS was from the existing DD2-based table family, and the same input, compiler configuration, and machine were retained while individual code changes were tested.

The working procedure was:

1. Compile one version of the solver.
2. Run the same 20-model sequence.
3. Measure wall-clock execution time using `/usr/bin/time -p`.
4. Save the generated primary output.
5. Apply exactly one optimisation.
6. Recompile and rerun.
7. Compare the old and new primary output using `diff`.
8. Keep the change only if the numerical output remained acceptable.

For the accepted *source-level* optimisations, the comparison

```
1 diff old_output.dat new_output.dat
```

returned no differences at the precision written to the output file.

This was important because a faster calculation is not useful if the optimisation changes the physical solution.

4 Compiler Optimisation Tests

Before changing the Fortran logic, different compiler optimisation levels were tested.

The final compilation command used for the optimised solver was

```
1 gfortran -O1 -fno-automatic \
2   -o logtov_seq_geom_tidal.out \
3   src/logtov_seq_geom_tidal.f90
```

4.1 -O1

The `-O1` level was retained as a conservative compiler optimisation. Relative to the unoptimised build, the observed numerical difference was extremely small: in the comparison performed during development, the only visible change was at approximately the 10^{-7} – 10^{-8} fractional level in the tidal coupling quantity κ_2 , while the key written k_2 and Λ values were unchanged in that benchmark.

4.2 Why -02 and -03 were rejected

Higher optimisation levels were also tested. In those builds, tidal quantities changed by roughly the percent level in the benchmark comparison (approximately 0.9% in the observed case).

Because the project requires tidal derivatives and curvature diagnostics, such a change was considered too large to accept merely for increased speed.

The likely origin of this sensitivity is the combination of default single-precision `real` arithmetic in parts of the original program and numerically sensitive cancellation in the tidal Love-number expressions. This interpretation should be treated as a numerical diagnosis rather than a formal proof.

Consequently,

-01 was chosen instead of -02/-03.

5 Source-Level Optimisations

5.1 Use the actual EoS table length

The interpolation routines previously used a generic array size when searching the EoS table. This was replaced by the actual number of loaded EoS entries:

```
1 nn = length
```

rather than searching over the full allocated array.

This change did not provide a significant standalone speed improvement, but it makes the interpolation search operate only on valid EoS entries and avoids unnecessary dependence on unused array storage.

It was therefore retained as a structural cleanup before the larger search optimisation.

5.2 Replace linear EoS searches with binary searches

This was the largest performance improvement.

The original interpolation routines located the bracketing EoS points by walking through the EoS array sequentially. Such a linear search has computational cost

$$\mathcal{O}(N)$$

for a table containing N points.

A binary-search helper,

```
1 find_upper_index(array, x, nused)
```

was introduced and used in the two principal interpolation directions:

- `EosShen2D`: search the pressure array when evaluating $\epsilon(P)$;

- **poutshen**: search the energy-density array when evaluating $P(\epsilon)$.

Binary search reduces the lookup cost to

$$\mathcal{O}(\log N).$$

This matters because EoS interpolation is performed repeatedly inside every RK4 stage and for every radial step of every stellar model.

Crucially, only the *search algorithm* was changed. Once the bracketing pair was found, the existing logarithmic interpolation was left unchanged.

The benchmark runtime decreased from approximately

$$11.46 \text{ s} \longrightarrow 3.30 \text{ s}.$$

Requirement

Binary search assumes that the searched EoS coordinate is monotonic. Therefore pressure and energy-density columns must be checked for monotonicity before an EoS is used with the solver.

5.3 Cache $\epsilon(P)$ in **rhsp**

The pressure-gradient routine **rhsp** used the same $\epsilon(P)$ value more than once while evaluating a single derivative.

Instead of repeatedly calling

```
1 eps(pp)
```

for the same pressure, the energy density was evaluated once and stored in a local variable, schematically:

```
1 ee = eps(pp)
2 ! use ee wherever eps(pp) was previously repeated
```

This removes duplicate interpolation calls while leaving the mathematical expression unchanged.

The benchmark changed from approximately

$$3.30 \text{ s} \longrightarrow 3.02 \text{ s}.$$

5.4 Cache the base $\epsilon(P)$ evaluation in **rhsb**

The tidal routine **rhsb** requires both the local energy density and an estimate of

$$\frac{d\epsilon}{dP}.$$

The original code estimates this derivative from nearby pressure values. The finite-difference evaluations at shifted pressures were deliberately left unchanged.

Only the repeated *central* call `eps(pp)` was cached and reused.

Conceptually, the calculation remains

$$\epsilon(P), \quad \epsilon(P + \delta P), \quad \epsilon(P - \delta P),$$

with the shifted EoS evaluations still performed independently. Thus the tidal finite-difference prescription itself was not modified.

The benchmark changed from approximately

$$3.02 \text{ s} \longrightarrow 2.36 \text{ s}.$$

5.5 Disable unnecessary diagnostic output

The original program generates auxiliary diagnostic files including

- `profile.dat`,
- `asshisto_tov.dat`.

These are not the primary TOV/tidal sequence output. Writing them during the radial integration introduces disk I/O, and the associated diagnostic logic also performed additional calculations, including repeated rest-density/EoS work.

For production sequence calculations these diagnostic operations were disabled.

The main output

`logtov_seq_geom_tidal.dat`

was retained unchanged.

The benchmark changed from approximately

$$2.36 \text{ s} \longrightarrow 2.17 \text{ s}.$$

If detailed radial profiles are needed for debugging, the diagnostic output can be re-enabled separately.

5.6 Cache repeated identical EoS queries in EosShen2D

A small cache was added to the energy-density interpolation routine. The routine stores the most recent pressure and its interpolated EoS result:

```
1 real, save :: last_pp = 0.0
2 real, save :: last_rrho = 0.0, last_e = 0.0
3 logical, save :: cache_valid = .false.
```

Before performing the logarithm, binary search, and interpolation, the routine checks whether the requested pressure is exactly the same as the previous request:

```

1 if (cache_valid .and. pp == last_pp) then
2   rrho = last_rrho
3   e = last_e
4   return
5 endif

```

After a normal interpolation the values are stored:

```

1 last_pp = pp
2 last_rrho = rrho
3 last_e = e
4 cache_valid = .true.

```

This is deliberately conservative. It does not approximate nearby pressures as equal; a cache hit occurs only for an exactly repeated floating-point input.

Repeated exact EoS requests occur because several differential-equation routines may ask for the same thermodynamic state during an RK stage.

The benchmark changed from approximately

$$2.17 \text{ s} \longrightarrow 1.63 \text{ s}.$$

6 Performance Summary

Table 1 summarises the measured optimisation sequence.

Table 1: Representative 20-model benchmark. Each row includes all optimisations above it.

Configuration	Runtime [s]	Speed-up vs. baseline
-O1, valid-table length used	11.46	1.00
Binary EoS search	3.30	3.47
Cache repeated $\epsilon(P)$ in <code>rhsp</code>	3.02	3.79
Cache base $\epsilon(P)$ in <code>rhsb</code>	2.36	4.86
Disable auxiliary diagnostic work	2.17	5.28
Cache identical <code>EosShen2D</code> requests	1.63	7.03

The total wall-clock reduction is

$$1 - \frac{1.63}{11.46} \simeq 0.858,$$

or approximately

$$\boxed{85.8\%}.$$

The most important individual optimisation was replacing the repeated linear EoS searches with binary searches.

7 What Was Deliberately Not Changed

Several tempting modifications were deliberately avoided because they would change the numerical problem rather than merely accelerate it.

1. **The radial step size was not increased.**

The original value

```
1 r0 = 10.*1e-5/1.746
2 h = 10.*1e-5/1.746
```

was retained. Although increasing h would reduce the number of radial steps and therefore runtime, it would also change integration resolution and should only be done as part of a convergence study.

2. **RK4 was not replaced.**

The fourth-order Runge–Kutta evolution and its update equations were left intact.

3. **The TOV equations were not modified.**

No physical term was removed from the stellar-structure equations.

4. **The tidal finite-difference prescription was not simplified.**

The shifted EoS evaluations required for the numerical estimate of $d\epsilon/dP$ were retained.

5. **The EoS interpolation formula was not changed.**

Interpolation remains linear in logarithmic EoS variables. Binary search only changes how the bracketing indices are found.

6. **The surface pressure criterion was not changed.**

The original integration termination logic was preserved.

This distinction is important: the optimisation targets computational overhead rather than changing the underlying numerical physics.

8 Numerical Equivalence Checks

Every accepted source optimisation was tested independently.

For the source-code changes listed in Section 5, the resulting primary output was identical under a direct file comparison at the precision written by the Fortran program.

In practice, the validation command was of the form

```
1 diff reference_20.dat optimized_20.dat
```

with an empty result indicating no written difference.

This gives a useful hierarchy of confidence:

- the algorithmic source optimisations preserved the written stellar sequence in the benchmark;

- -01 introduced only a very small compiler-level numerical difference in the specific comparison used during development;
- -02 and -03 were rejected because the change in tidal quantities was too large.

The tests establish equivalence for the benchmark runs used during development. They do not replace standard convergence studies when new EoS families or derivative-sensitive observables are introduced.

9 Why the Original Code Was Slow

The runtime was not dominated by one unusually complicated physical equation. Instead, the major cost came from the number of times relatively small pieces of work were repeated.

A single RK4 radial step evaluates several right-hand-side routines at four RK stages. Those routines repeatedly require EoS values. A full stellar sequence repeats this operation over many radial points and many central densities.

Schematically,

$$N_{\text{work}} \sim N_{\text{models}} \times N_{\text{radial}} \times N_{\text{RK}} \times N_{\text{EoS queries}}.$$

Therefore even a modest amount of wasted work inside an EoS query becomes expensive after being multiplied by the number of calls.

The original sequential EoS scan was especially costly:

$$\boxed{\text{many RK evaluations} \times \mathcal{O}(N_{\text{EoS}}) \text{ search}}$$

whereas the optimised version uses

$$\boxed{\text{many RK evaluations} \times \mathcal{O}(\log N_{\text{EoS}}) \text{ search.}}$$

Caching then removes a subset of those searches entirely.

10 Full-Sequence Behaviour

After the optimisation, a 350-model sequence using one of the original DD2-based EoSs completed in approximately

$$206.7 \text{ s}$$

(~ 3.45 minutes) in the test performed during development.

This full-sequence runtime should not be interpreted as a universal benchmark because it depends on

- the EoS,

- the central-density range,
- the number of requested models,
- how many radial steps are needed before the surface is reached,
- compiler version,
- and hardware.

It nevertheless demonstrated that the optimised code could be used for large EoS scans without changing the radial step purely for performance.

11 Constraints and Remaining Numerical Issues

11.1 Monotonic EoS requirement

The binary-search routines require the relevant EoS arrays to be monotonic. Every new EoS should therefore be validated before production runs. In particular,

$$\frac{dP}{d\epsilon} > 0$$

must not be violated by an accidental table ordering or a crust–core splice artefact.

11.2 Derivative-sensitive observables

The final science analysis includes curvature and derivatives of tidal quantities. These amplify numerical noise. Therefore identical single-run output does not remove the need for resolution studies in central density when computing

$$\frac{dR}{dM}, \quad \frac{d^2R}{dM^2}, \quad \frac{d\Lambda}{dM}, \quad \frac{d^2\lambda}{dM^2}.$$

11.3 Known correctness issue kept separate from optimisation

During code inspection, a potential issue was noted in `rhsn`: the routine appears to use global `r` and `m` values internally rather than exclusively using its trial arguments `rr` and `mm`.

This was **not** altered during the optimisation work.

The reason for leaving it untouched was methodological: performance changes and physics/correctness changes should not be mixed in the same validation step. It should instead be investigated separately with an independent regression and convergence test.

12 Final Recommended Build

The accepted performance build is

```

1 gfortran -O1 \
2   -fno-automatic \
3   -o logtov_seq_geom_tidal.out \
4   src/logtov_seq_geom_tidal.f90

```

A run can then be timed with

```

1 /usr/bin/time -p ./logtov_seq_geom_tidal.out

```

and a regression comparison can be performed with

```

1 cp logtov_seq_geom_tidal.dat current_run.dat
2 diff reference_run.dat current_run.dat

```

For future modifications, the same one-change-at-a-time procedure should be retained.

13 Optimisation Timeline

The final sequence of accepted changes was:

1. Adopt conservative compiler optimisation `-O1`.
2. Restrict interpolation searches to the actual loaded table length.
3. Replace linear EoS scans with binary search.
4. Reuse the local $\epsilon(P)$ evaluation inside `rhsp`.
5. Reuse the central $\epsilon(P)$ evaluation inside `rhsb`.
6. Remove unnecessary profile/histogram diagnostic work from production runs.
7. Add a last-value cache to `EosShen2D` for exactly repeated pressure requests.

The cumulative result was approximately

$$11.46 \text{ s} \rightarrow 1.63 \text{ s}$$

for the representative 20-model benchmark, while preserving the written primary output through the accepted source-level changes.

14 Conclusion

The principal lesson from the optimisation is that the TOV and tidal equations themselves did not need to be simplified to obtain a large speed improvement.

Most of the gain came from reducing repeated EoS-search work inside the RK4 loop:

$$\text{linear search} \rightarrow \text{binary search} \rightarrow \text{reuse repeated values.}$$

Additional savings came from removing diagnostic I/O and redundant calculations that were not required for the primary sequence output.

The final implementation is approximately seven times faster in the representative benchmark while retaining the original radial resolution, RK4 integrator, physical equations, EoS interpolation prescription, and tidal finite-difference structure.

This provides a substantially more practical base for calculations over large hyperonic EoS ensembles and for the high-resolution stellar sequences required by derivative-based analyses.